

Fecha de elaboracion: 2026-05-14

Repositorio: Sugarbayweb

Rama auditada: 'sheyla/front-theme'

Commit de referencia: '15d1d8451557ee0d3722a09891f532dd2bee24b5' (mensaje: 'feature-front')

1. Objetivo funcional de la rama

Esta rama implementa la primera base visual y funcional del front-end retro para la seccion de conciertos y para el header. El objetivo principal fue dejar de tener una UI generica y mover la experiencia a un lenguaje visual Windows 95 / retro synthwave, manteniendo la navegacion y la logica de negocio.

2. Evidencia Git y trazabilidad

- Autor del commit: 'root <you@example.com>'.
- Fecha commit: '2026-04-29T14:58:11+02:00'.
- Archivos tocados: 14.
- Churn total: 1.306 lineas anadidas y 323 eliminadas.
- Integracion: el commit esta contenido en 'main' y en las ramas 'sheyla/front-end' y 'sheyla/new-changes'.

Archivos modificados:

- 'app/globals.css'
- 'app/layout.tsx'
- 'components/concerts/concert-cards-client.tsx'
- 'components/concerts/concert-filters.tsx'
- 'components/concerts/concert-pagination.tsx'
- 'components/concerts/concerts-catalog-page.tsx'
- 'components/layout/header-client.tsx'
- 'components/ui/page-hero.tsx'
- 'lib/repositories/concerts.ts'
- 'lib/repositories/safe-query.ts'
- 'package-lock.json'
- 'public/images/hero-concerts.png'
- 'dev-server.err.log' y 'dev-server.out.log' (artefactos de entorno)

3. Resumen cuantitativo de cambios

Top de ficheros por volumen de cambio:

- 'app/globals.css': +647 / -5 (652)
- 'components/concerts/concert-filters.tsx': +149 / -80 (229)
- 'components/concerts/concert-cards-client.tsx': +124 / -85 (209)
- 'components/layout/header-client.tsx': +92 / -50 (142)
- 'lib/repositories/concerts.ts': +124 / -6 (130)

Lectura tecnica: la rama no se limito a "skin" visual; combina una capa estetica grande con cambios reales de interaccion y consulta de datos en conciertos.

4. Cambios implementados en detalle

4.1 Sistema visual retro base (app/globals.css)

Se introduce un sistema de estilos reusable con variables y clases de componente:

- Variables CSS de paleta y tipografia: '--sb-bg-main', '--sb-surface-*', '--sb-accent-*', '--font-retro-ui', '--font-retro-pixel'.
- Clases de ventana/boton/input estilo Win95:
 - '.win-window'
 - '.win-titlebar'
 - '.win-button'
 - '.win-input'
- Clases de tarjetas y paginacion de conciertos:
 - '.retro-concert-card'
 - '.retro-concert-header'
 - '.retro-card-action'
 - '.retro-pagination-*'
- Clases de modal de filtros y overlay:
 - '.retro-filters-modal'
 - '.retro-modal-overlay'
 - '.retro-folder-button'

Impacto:

- Se normaliza el look retro en una sola hoja global.
- Se habilita reutilizacion posterior en ramas siguientes.

4.2 Hero de conciertos (components/ui/page-hero.tsx)

Se crea un hero visual con 'next/image' ('/images/hero-concerts.png') y overlay oscuro para legibilidad:

- Fondo con imagen full-cover.
- Overlay 'bg-black/40'.
- Texto centrado con jerarquia 'eyebrow > title > description'.

Impacto:

- Primer encabezado visual consistente de seccion.
- Mejora del contraste y legibilidad sobre imagen.

4.3 Catalogo de conciertos (components/concerts/concerts-catalog-page.tsx)

Cambios estructurales:

- Introduccion de 'PageHero' al inicio.
- Barra superior de controles retro con:
 - 'ConcertFilters' en modo 'folder-modal'.
 - 'ConcertPagination' personalizada.
- Soporte de navegacion entre upcoming/past con CTA final.

Impacto funcional:

- Se mantiene la logica de filtros y paginacion.
- Se cambia el punto de entrada visual de los controles (mas accesible y compacto).

4.4 Filtros con modalidad panel/modal (components/concerts/concert-filters.tsx)

Implementacion clave:

- Prop 'mode' con dos variantes: 'panel' y 'folder-modal'.
- Estado local 'isOpen' para modal.
- Cierre por 'Escape' y por click en overlay.
- Bloqueo de scroll de 'body' mientras el modal esta abierto.
- Conserva 'form method="get"' para que los filtros sigan operando por query params.

Impacto:

- Experiencia mas limpia en pantallas reducidas sin perder semantica ni comportamiento.

- Mejor control de foco contextual para usuario.

4.5 Cards y modal de detalle de concierto (components/concerts/concert-cards-client.tsx)

Cambios implementados:

- Rediseño completo de cards con metadatos de fecha, venue y descripción.
- Iconografía SVG retro integrada.
- Modal de detalle por card controlado con 'selectedConcertId'.
- Cierre por 'Escape' y overlay.
- Bloqueo temporal del scroll de fondo al abrir modal.
- Acciones condicionales por tipo de concierto:
- 'upcoming': compra/info según 'actionUrl'.
- 'past': accesos a fotos/videos si existen.

Impacto funcional:

- Mejora del descubrimiento de información sin navegar a otra página.
- Conserva rutas y enlaces existentes.

4.6 Paginación retro (components/concerts/concert-pagination.tsx)

Se implementa paginación con:

- Flechas prev/next.
- Ventana de páginas visibles ('getVisiblePages').
- Serialización de filtros sin perder query previa.

Impacto:

- Navegación paginada más compacta y visualmente integrada.

4.7 Repositorio de conciertos robusto (lib/repositories/concerts.ts)

Cambios de datos relevantes:

- Construcción de filtros por fecha y país con reglas distintas para 'upcoming' y 'past'.
- Generación de 'googleMapsUrl' por concierto.
- Fallback local de conciertos cuando falla DB.
- Enriquecimiento de contenido:
- Matching de álbums/fotos/videos por ciudad, slug y proximidad temporal.
- Selección de foto de venue cuando existe.

Impacto técnico:

- Mejor resiliencia si Prisma/DB no responde.
- Mayor valor de contenido por concierto sin cambiar contratos de UI.

4.8 Fallback transversal a errores de base de datos (lib/repositories/safe-query.ts)

Se crea 'withDatabaseFallback' para capturar errores recuperables de Prisma ('P1001', 'P1017', 'P2021', 'P2022') y devolver datos de respaldo.

Impacto:

- Evita caídas completas de front ante indisponibilidad parcial de DB.
- Facilita demos en entorno local/inestable.

5. Tecnologías y patrones usados en la rama

- Framework: Next.js App Router (TypeScript).
- Estilos: Tailwind + capa CSS global personalizada retro.
- Interactividad: React Client Components con 'useState', 'useEffect', 'useMemo'.

- Datos: repositorios Prisma con fallback seguro.
- Assets: 'next/image' + imagen local en 'public/images'.

Patrones relevantes:

- Separacion presentacion/logica (componentes vs repositorios).
- Reutilizacion de clases visuales ('win-*', 'retro-*').
- Query params como contrato para filtros/paginacion.

6. Responsive, UX y accesibilidad en esta rama

Responsive detectado en codigo:

- Grids adaptativos ('grid-cols-1', 'md:grid-cols-2', 'lg:grid-cols-3').
- Controles con modal para modo compacto ('folder-modal').

Accesibilidad aplicada:

- 'aria-label' en botones de cierre/acciones.
- 'role="dialog"' y 'aria-modal="true"' en modales.
- Cierre por teclado ('Escape').
- Contrastes reforzados en titlebars y botones.

7. Riesgos o deuda tecnica identificada

- Se versionaron logs de desarrollo ('dev-server.*.log'), no deberian formar parte funcional de una rama de front.
- 'app/globals.css' crece de forma intensa; conviene modularizar en fases posteriores para reducir deuda de mantenimiento.
- No se detectaron tests automaticos asociados a estos cambios.

8. Validaciones realizadas para este informe

- Inspeccion directa de commit con:
- 'git show --name-status --stat'
- 'git show --numstat'
- Revision manual de archivos TSX/TS/CSS del commit

No se pudo ejecutar 'npm run lint' en este entorno porque 'npm' no esta disponible en la sesion actual.

9. Como defender oralmente esta rama

9.1 Version de 1 minuto

"En 'sheyla/front-theme' construimos la base visual retro del proyecto: sistema de colores, tipografias y componentes tipo ventana Win95. Ademas de estilo, mejoramos la experiencia de conciertos con filtros en modal, paginacion propia y un modal de detalle por concierto. A nivel tecnico, anadimos fallback de base de datos para que la web sea mas robusta en demo y en entornos con incidencias."

9.2 Version tecnica de 3 minutos

"La rama toca 14 archivos y mueve 1.629 lineas de churn. El nucleo es 'app/globals.css', donde se definen tokens visuales y componentes reutilizables ('win-window', 'win-titlebar', 'win-button'). En 'components/concerts', reestructuramos el catalogo: 'concert-filters' introduce doble modo panel/modal con bloqueo de scroll y cierre por Escape; 'concert-cards-client' incorpora detalle en dialogo, acciones condicionales upcoming/past y enlaces de contexto; 'concert-pagination' encapsula serializacion de query params. En datos, 'lib/repositories/concerts.ts' integra reglas de filtrado temporal y

mapeo de recursos multimedia por concierto. Para estabilidad, 'withDatabaseFallback' captura errores recuperables de Prisma y evita que la UI se rompa por caídas de DB."

9.3 Preguntas típicas y respuestas

- Pregunta: "Que parte fue solo visual y que parte funcional?"

Respuesta: "Visualmente, 'globals.css', 'page-hero' y clases retro. Funcionalmente, filtros/modales de conciertos, paginación y fallback de repositorio."

- Pregunta: "Como mantuviste accesibilidad en modales?"

Respuesta: "Usamos 'role=dialog', 'aria-modal', botones con 'aria-label', cierre por Escape y bloqueo de scroll del fondo."

- Pregunta: "Que riesgo dejaste abierto?"

Respuesta: "La hoja global creció bastante y hay logs versionados; lo razonable es modularizar CSS y limpiar artefactos de entorno."

10. Conclusiones de la rama

'sheyla/front-theme' es una rama fundacional: establece lenguaje visual reutilizable y refuerza la UX de conciertos con patrones de interacción retro coherentes. También introduce una capa de resiliencia en acceso a datos que mejora la estabilidad de la aplicación en escenarios de error.